DBpedia into Neo4j

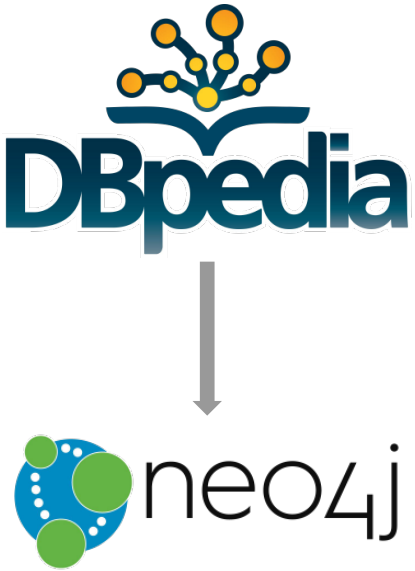
Big Data Technologies Course

a.a. 2016/2017

Federico Bianchi, Marco Cremaschi, Alessandro Gabrielli

Main Goals of the Project

- Import DBpedia into Neo4j
- Evaluate the performance of Neo4j
- Comparison of performance between Neo4j and Virtuoso

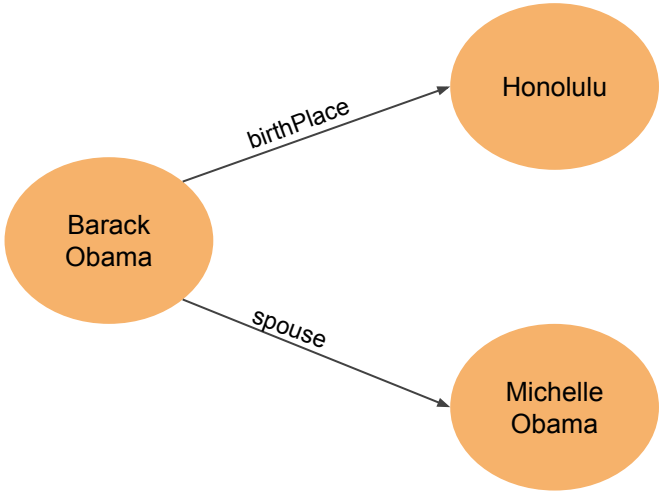


Process

1. Re-elaboration of the DBpedia structure
    a. Download DBpedia
    b. Using a Scala script to convert DBpedia
    c. Create a CSV
2. Importation of DBpedia into Neo4j
3. Creation of a cluster
4. Installation of Neo4j in the cluster
5. Performance evaluation

DBpedia

DBpedia is the structured representation of Wikipedia



Triple Structure: Subject - Predicate - Object

Neo4j

Neo4j is an industrial scale graph database:

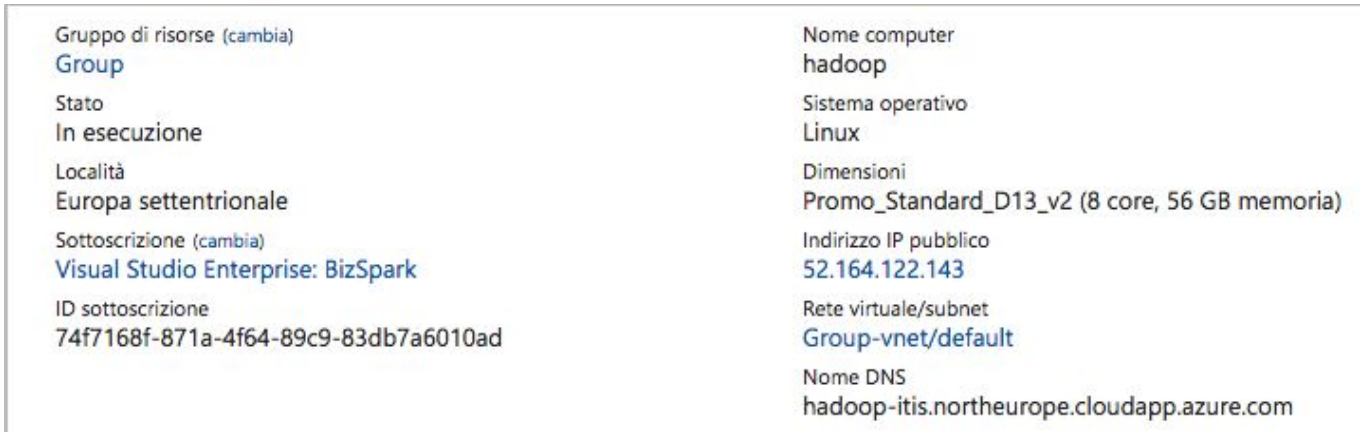
1. Fast for traversal queries
2. Provides REST APIs
3. Simple query language

Is it possible to import DBpedia data into Neo4j?

1. Re-elaboration of the DBpedia structure

Step 1: Server specifications

Microsoft Azure



Neo4j 3.1.3
Hadoop 2.7.3
SBT 0.13.15
Scala 2.10.6

Step 1a: Download DBpedia

FILE: DBpedia URI mapped to Wikipedia URI - wikipedia_links_en.nt
HEADER: DBPEDIA_RESOURCE_URI, RDF_TYPE, WIKIPEDIA_PAGE_URI
<http://dbpedia.org/resource/AccessibleComputing> <http://xmlns.com/foaf/0.1/isPrimaryTopicOf>
<http://en.wikipedia.org/wiki/AccessibleComputing> .

FILE: Wikipedia page link - page_links_en.nt
HEADER: DBPEDIA_RESOURCE_SRC, RDF_TYPE, DBPEDIA_RESOURCE_DST
<http://dbpedia.org/resource/AccessibleComputing> <http://dbpedia.org/ontology/wikiPageWikiLink>
<http://dbpedia.org/resource/Computer_accessibility> .

FILE: Page titles mapped to DBpedia URI - labels_en.nt
HEADER: DBPEDIA_RESOURCE_URI, RDF_TYPE, PAGE_NAME
<http://dbpedia.org/resource/AccessibleComputing> <http://www.w3.org/2000/01/rdf-schema#label> "AccessibleComputing"@en .

Step 1a: Download DBpedia

FILE: DBpedia categories mapped to pages - article_categories_en.nt
HEADER: DBPEDIA_RESOURCE_URI, RDF_TYPE, DBPEDIA_CATEGORY_URI
<http://dbpedia.org/resource/Albedo> <http://purl.org/dc/terms/subject> <http://dbpedia.org/resource/Category:Climate_forcing> .

FILE: DBpedia ontology mapped to pages - instance_types_en.nt
HEADER: DBPEDIA_RESOURCE_URI, RDF_TYPE, DBPEDIA_ONTOLOGY_URI
<http://dbpedia.org/resource/Autism> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://dbpedia.org/ontology/Disease> .

Step 1a: Scala script workflow

Since Neo4j can import files with a CSV extension, we had to convert the DBpedia datasets. In order to convert the dataset and create the CSV file, we used a Scala script (https://github.com/kbastani/neo4j-dbpedia-importer)
The script uses Spark to process files and Hadoop to save them

Step 1b: Scala script workflow

1 **Import the page nodes and link graph**

1. First stage: Join the Wikipedia map file and the names map file into a single RDD
   a. Extract Wikipedia links from wikipedia_links_en.nt
   b. Extract Pages name from page_links_en.nt
   c. Join Wikipedia links and Pages name
   d. Take the union of the two datasets and generate a CSV
2. Second stage: Encode each value in the page links file with the unique node id generated during the last stage
   a. Create an in-memory hash table to lookup DBpedia keys and return the encoded unique node id
   b. Process page links saved in page_links_en.nt
3. Final stage: Save the page nodes and relationship results to HDFS

Step 1b: Scala script workflow

2 **Import the category nodes**

1. Extract categories from article_categories_en.nt
   a. Get a distinct list of categories and generate a node index
   b. Save the category list to HDFS

Step 1b: Scala script workflow

3 **Import the ontology graph**

1. Extract ontology types from instance_types_en.nt
   a. Get a distinct list of ontology and generate a node index
   b. Save the ontology list to HDFS

Step 1b: Output

HDFS dir: /pagenodes
HEADER: dbpedia, id, l:label, wikipedia, title
EXAMPLE: http://dbpedia.org/resource/Guy_C._Swan 177 Page http://en.wikipedia.org/wiki/Guy_C._Swan Guy C. Swan
HDFS dir: /pagerels
HEADER: start, end, type
EXAMPLE: 2803179 9838885 HAS_LINK
HDFS dir: /categorynodes
HEADER: dbpedia, id, l:label
EXAMPLE: 10995010 Category http://dbpedia.org/resource/Category:Corruption_in_Chile Corruption in Chile
HDFS dir: /categoryrels
HEADER: start, end, type
EXAMPLE: 10994968 11144781 HAS_CATEGORY
HDFS dir: /ontologynodes
HEADER: dbpedia, id, l:label, wikipedia, title
EXAMPLE: 11855115 Ontology http://schema.org/Book Book
HDFS dir: /ontologyrels
HEADER: start, end, type
EXAMPLE: 11855053 8324598 HAS_ONTOLOGY

Step 1c: Create a CSV

```
File name:  page_nodes.csv
Command:    bin/hadoop fs -cat "/pagenodes/part-" | bin/hadoop fs -put - /page_nodes.csv
File name:  page_rels.csv
Command:    bin/hadoop fs -cat "/pagerels/part-" | bin/hadoop fs -put - /page_rels.csv
File name:  category_nodes.csv
Command:    bin/hadoop fs -cat "/categorynodes/part-" | bin/hadoop fs -put - /category_nodes.csv
File name:  category_rels.csv
Command:    bin/hadoop fs -cat "/categoryrels/part-" | bin/hadoop fs -put - /category_rels.csv
File name:  ontology_nodes.csv
Command:    bin/hadoop fs -cat "/ontologynodes/part-" | bin/hadoop fs -put - /ontology_nodes.csv
File name:  ontology_rels.csv
Command:    bin/hadoop fs -cat "/ontologyrels/part-" | bin/hadoop fs -put - /ontology_rels.csv
```

2. Import of DBpedia into Neo4j

Step 2: Import DBpedia (first try)

```
USING PERIODIC COMMIT
LOAD CSV WITH HEADERS FROM 'file:///ontology_nodes.csv' AS line
FIELDTERMINATOR '\t'
CREATE (:Ontology { ontologyId: line.id, label: line.label, dbpedia: line.dbpedia, title: line.title})

USING PERIODIC COMMIT
LOAD CSV WITH HEADERS FROM 'file:///category_nodes.csv' AS line
FIELDTERMINATOR '\t'
CREATE (:Category { ontologyId: line.id, label: line.label, dbpedia: line.dbpedia, title: line.title})

USING PERIODIC COMMIT
LOAD CSV WITH HEADERS FROM 'file:///page_nodes.csv' AS line
FIELDTERMINATOR '\t'
CREATE (:Page { ontologyId: line.id, label: line.label, dbpedia: line.dbpedia, title: line.title})

CREATE INDEX ON :Ontology(ontologyId)

...
```

TOT: 6h!

Step 2: Import DBpedia (first try)

```
USING PERIODIC COMMIT
LOAD CSV WITH HEADERS FROM 'file:///ontology_rels.csv' AS line
FIELDTERMINATOR '\t'
MATCH (s:Ontology {ontologyId: line.start}), (e:Page {pageId: line.end})
CREATE (s)-[:HAS_ONTOLOGY]->(e);
```

TOT: 6h!

Step 2: Import DBpedia (second try)

```
neo4j-import --into dbpedia.db --delimiter '\t' --id-type string \
--nodes:Ontology ontology_nodes_test.csv \
--nodes:Category category_nodes_test.csv \
--nodes:Page page_nodes_test.csv \
--relationships:HAS_ONTOLOGY ontology_rels_test.csv \
--relationships:HAS_CATEGORY category_rels_test.csv \
--relationships:HAS_LINK page_rels_test.csv
```

TOT: 15 minutes!

Step 2: Import DBpedia using Neo4J Command line

FILE: ontology_nodes.csv
HEADER - ontologyId:ID(Ontology), label, dbpedia, title

FILE: category_nodes.csv
HEADER - categoryId:ID(Category), label, dbpedia, title

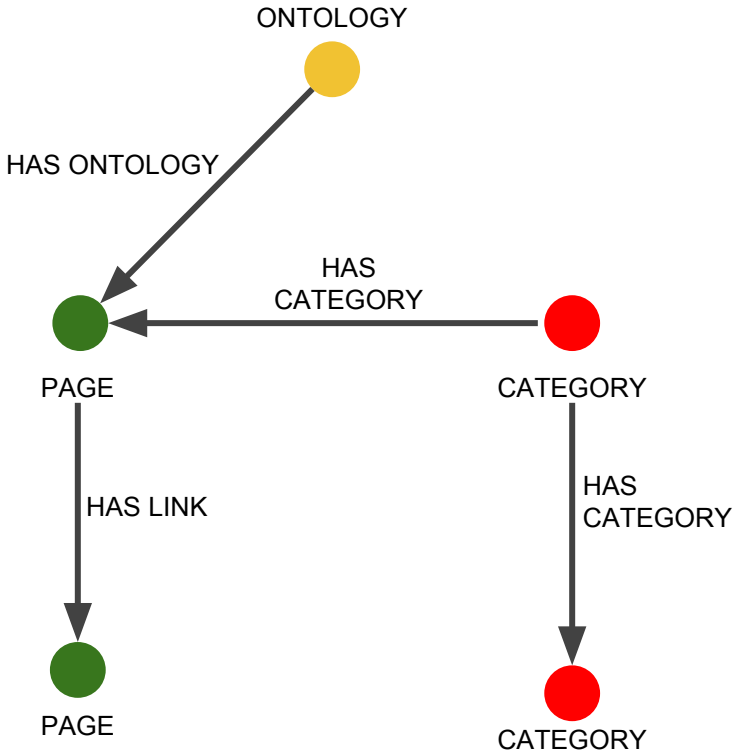
FILE: pages_nodes.csv
HEADER - pageId:ID(Page), label, wikipedia, title

FILE: ontology_rels.csv
HEADER - :START_ID(Ontology), :END_ID(Page), type:IGNORE

FILE: category_rels.csv
HEADER - :START_ID(Category), :END_ID(Category), type:IGNORE
HEADER - :START_ID(Category), :END_ID(Page), type:IGNORE

FILE: page_rels.csv
HEADER - :START_ID(Page), :END_ID(Page), type:IGNORE

Step 2: Neo4j Representation



We imported:

~10.000.000 Nodes
~100.000.000 Relations

3. Creation of a cluster

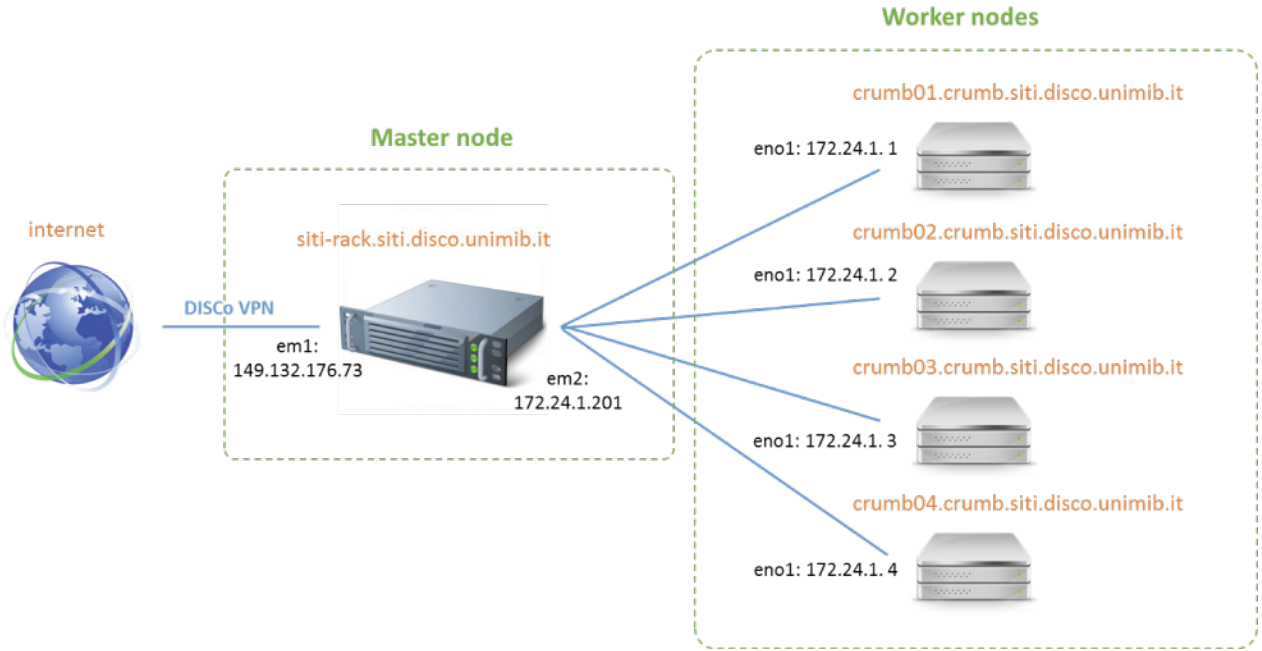
Step 3: Cluster Specifications

MASTER NODE	
CPU	2 x Intel Xeon X5550 quad-core (4 core fisici – 8 core virtuali) – 8 MB cache L2, 2.66 GHz (Turbo Boot 3.06 GHz) – TDP 95 W
RAM	32 GB DDR3 800 MHz ECC
System Storage	2 x HD SAS 140 GB 15k RPM in RAID 1 hardware
Data Storage	4 x HD SAS 500 GB 10k RPM in RAID 5 hardware
Network	2 x Gigabit Ethernet
Model year	2009

Step 3: Cluster Specifications

Worker Nodes	
CPU	1 x Intel Core i7-6770HQ quad-core (4 core fisici – 8 core virtuali) – 6 MB cache L3, 2.6 GHz (turbo boost 3.5 GHz) – TDP 45 W
RAM	32 GB DDR4 2133 MHz non-ECC
Storage	1 x SSD M.2 (SATA 6Gbps) 1 TB
Network	1 x Gigabit Ethernet
Model year	2016

Step 3: Cluster Specifications



Step 3: Cluster Specifications

Cloudera Distribution Including Apache Hadoop v 5.10.1

- Apache Hadoop 2 (YARN, HDFS, MapReduce v2)
- Apache Hive, Apache Pig, Apache Spark, Apache Parquet, …
- Cloudera Manager

4. Installation of Neo4j in a Cluster

Step 4: Configuration

Following the guidelines in
http://neo4j.com/docs/operations-manual/current/tutorial/highly-available-cluster//
we installed Neo4j on 2 machines. Data replication is thus easily achievable

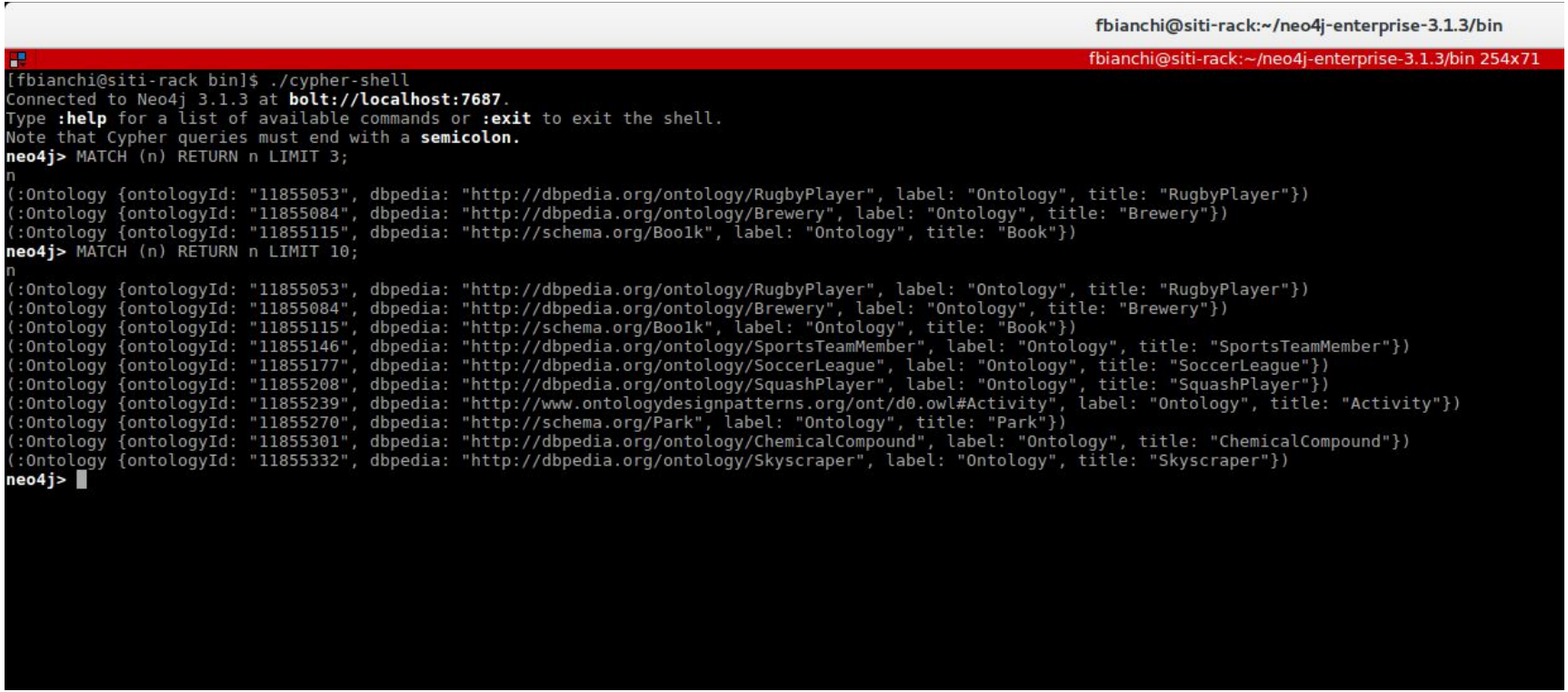
```
# Unique server id for this Neo4j instance
# can not be negative id and must be unique
ha.server_id = 1

# List of other known instances in this cluster
ha.initial_hosts = neo4j-01.local:5001,neo4j-02.local:5001
# Alternatively, use IP addresses:
#ha.initial_hosts = 192.168.0.20:5001,192.168.0.21:5001,192.168.0.22:5001

# HA - High Availability
# SINGLE - Single mode, default.
dbms.mode=HA

# HTTP Connector
dbms.connector.http.enabled=true
dbms.connector.http.listen_address=:7474
```

Step 4: Neo4j with DBpedia on Cluster



5. Performance Evaluation

Experimental Settings

We tested some queries on both Neo4j and Virtuoso; we then computed the time needed to obtain results

Virtuoso is a database engine hybrid that allows to store RDF data, querying is possible using the SPARQL language.

Each query has been run 10 times and average the results

We report the most meaningful results and insights

Server choice

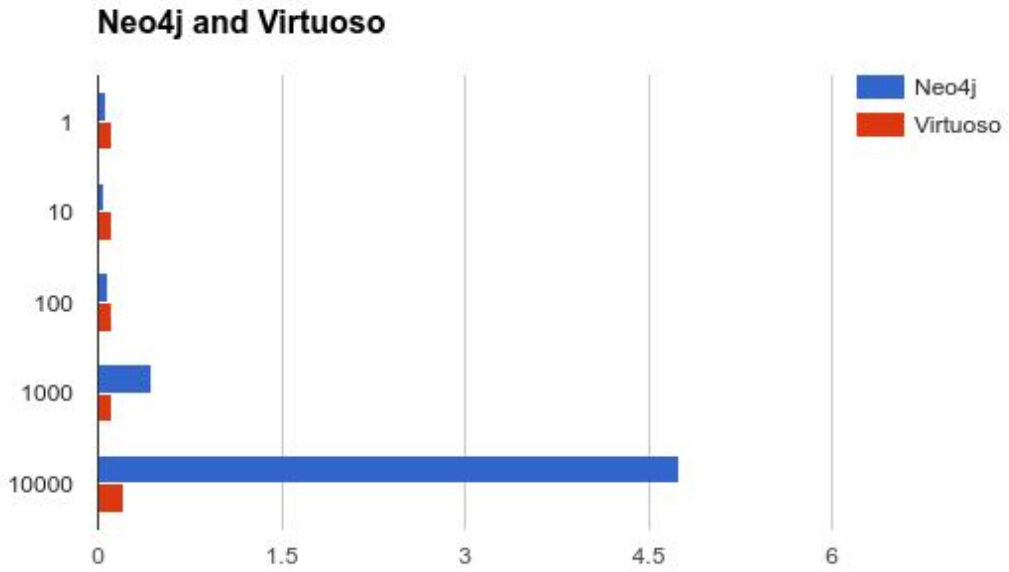
We decided to test the performance on the Azure server we used for the preceding steps because:

1. Azure server is faster and is easily accessible http://hadoop-itis.northeurope.cloudapp.azure.com:7474/browser/
2. Some queries on the cluster required more than 15 minutes

Comparison with Limit (average in seconds)

MATCH (n) RETURN(n)
LIMIT K

SELECT ?entity WHERE {
?entity ?a ?b } LIMIT K



Query Performance

Triples in which the entity Barack Obama is subject

MATCH (n:Page { dbpedia: 'http://dbpedia.org/resource/Barack_Obama' })-[r]->(k) RETURN n, r, k LIMIT 100

average: 2.63 seconds **standard deviation**: 0.08 seconds

maximum: 2.81 seconds **minimum**: 2.55 seconds

SELECT ?a, ?b WHERE { <http://dbpedia.org/resource/Barack_Obama> ?a ?b } LIMIT 100

average: 0.11 seconds **standard deviation**: 0.002 seconds

maximum: 0.12 seconds **minimum**: 0.10 seconds

Better

Query Performance

Number of triples in which the entity Barack Obama is subject

MATCH (n:Page { dbpedia: 'http://dbpedia.org/resource/Barack_Obama' })-[r]->(k) RETURN (count(r))

average: 10.56 seconds **standard deviation**: 1.62 seconds

maximum: 15.14 seconds **minimum**: 9.81 seconds

SELECT count(?a) WHERE { <http://dbpedia.org/resource/Barack_Obama> ?a ?b }

average: 0.14 seconds **standard deviation**: 0.04 seconds

maximum: 0.24 seconds **minimum**: 0.11 seconds

Better

Schema in Neo4j

We realized that id indexing is not suitable for query that make heavy use of strings/pattern matching

We thus tested (rewrote) the queries using the *hardcoded* ids of the entities

Query Performance using IDs

1) Triples in which the entity Barack Obama is subject
2) Number of triples in which the entity Barack Obama is subject

MATCH (n:Page)-[r]->(k)WHERE ID(n) = 3483224 RETURN n, r, k LIMIT 100

average: 0.15 seconds **standard deviation**: 0.02 seconds

maximum: 0.22 seconds **minimum**: 0.13 seconds

Neo4j average Before: 2.63 seconds
Virtuoso Average: 0.11 seconds

MATCH (n:Page)-[r]->(k) WHERE ID(n) = 3483224 RETURN (count(r))

average: 0.09 seconds **standard deviation**: 0.09 seconds

maximum: 0.36 seconds **minimum**: 0.05 seconds

Neo4j average Before: 10.56 seconds
Virtuoso Average: 0.15 seconds

Paths in Neo4j

Paths between Barack Obama and Michelle Obama with 2 hop

```
MATCH p=(p1:Page {dbpedia:'http://dbpedia.org/resource/Barack_Obama'})-[*..2]-(p3:Page
{dbpedia:'http://dbpedia.org/resource/Michelle_Obama'}) RETURN p
```

average: 87.78 seconds **standard deviation**: 23.74 seconds

maximum: 153.83 seconds **minimum**: 73.76 seconds

Paths in Neo4j using IDs

Paths between Barack Obama and Michelle Obama with 2 hops

MATCH p=(p1:Page)-[*..2]-(p3:Page)

WHERE ID(p3) = 3483224 AND ID(p1) = 1023097

RETURN p

average: 22.47 seconds **standard deviation**: 7.83 seconds

maximum: 41.05 seconds **minimum**: 16.85 seconds

Average Before: 87.78

Schema in Neo4j

We can use INDEX to optimize the retrieval.

```
CREATE INDEX ON :Page(dbpedia)
```

This comes at the cost of additional storage space and slower writes, so deciding what to index and what not to index is an important and often non-trivial task

http://neo4j.com/docs/developer-manual/current/cypher/schema/index/

Example of improvement using an index

Number of triples in which the entity Barack Obama is subject

```
MATCH (n:Page { dbpedia: 'http://dbpedia.org/resource/Barack_Obama' })-[r]->(k)
RETURN (count(r))
```

average: 0.05 seconds **standard deviation**: 0.004 seconds

maximum: 0.06 seconds **minimum**: 0.046 seconds

Neo4j average Before (using Ids): 0.15 seconds
Virtuoso Average: 0.09 seconds

Considerations on performance

Schema representation and Indexing in Neo4j are fundamental to obtain good performance

The index in Neo4j should be the URI of the resource, the use of integer IDs should be avoided

Querying using IDs and the Index on Neo4j has performance similar to the ones obtained with Virtuoso

Neo4j required a bit of configuration while importing data in Virtuoso was an out-of-the-box operation.

Thank you

Query da inserire

MATCH (n:Page)-[r]->(k)WHERE ID(n) = 3483224 RETURN n, r, k LIMIT 1

mean: 0.0920258098178 sd: 0.0864017845943

max: 0.336222171783 min: 0.0550320148468

MATCH (n:Page)-[r]->(k)WHERE ID(n) = 3483224 RETURN n, r, k LIMIT 10

mean: 0.0687176386515 sd: 0.00685864008879

max: 0.0794770717621 min: 0.0578689575195

MATCH (n:Page)-[r]->(k)WHERE ID(n) = 3483224 RETURN n, r, k LIMIT 100

mean: 0.158081849416 sd: 0.0269740002553

max: 0.228208065033 min: 0.134288072586

Paths in Neo4j

Paths between Barack Obama and Michelle Obama with 1 hop

```
MATCH p=(p1:Page {dbpedia:'http://dbpedia.org/resource/Barack_Obama'})-[*..1]-(p3:Page
{dbpedia:'http://dbpedia.org/resource/Michelle_Obama'}) RETURN p
```

average: 10.24 seconds **sd**: 0.24 seconds

max: 10.81 seconds **min**: 9.92 seconds

Paths in Neo4j using IDs

Paths between Barack Obama and Michelle Obama with 1 hop

```
MATCH p=(p1:Page)-[*..1]-(p3:Page)

WHERE ID(p3) = 3483224 AND ID(p1) = 1023097

RETURN p
```

average: 0.09 seconds **standard deviation**: 0.14 seconds

maximum: 0.50 seconds **minimum**: 0.04 seconds

Average Before: 10.24